
Reducing the Cost of Muon with Monarch-Parameterized Layers

Ruslan Kabirov
MIPT
Moscow, Russia
kabirov.ra@phystech.edu

Alexandr Shestakov
MIPT
Moscow, Russia
ashestakov@phystech.edu

Abstract

Muon has recently gained attention as an optimizer for neural network training because of its use of orthogonalized matrix-valued updates [1, 2]. In its standard form, Muon relies on Newton–Schulz iterations to approximate the polar factor of a matrix update, which introduces additional computational overhead. We study whether this cost can be reduced by replacing selected dense layers with Monarch-parameterized layers [3]. This structure allows Newton–Schulz iterations to be applied to smaller block matrices and creates opportunities for parallel block-wise computation. We evaluate the proposed variants in pretraining experiments with a compact GPT-2 style model and compare them against Muon and AdamW baselines. Our goal is to reduce the cost of Muon’s orthogonalization step while keeping validation loss and perplexity close to the baseline methods.

Keywords: Muon optimizer, Monarch matrices, matrix orthogonalization, Newton–Schulz iteration, polar decomposition, structured matrices, computational efficiency.

1 Introduction

1.1 Muon and orthogonalized updates

Muon was introduced as an optimizer for hidden layers in neural networks and has recently demonstrated strong empirical performance in deep learning [1]. More recent work has shown that Muon can scale to large language model training when combined with appropriate update scaling and weight decay choices [2]. Unlike standard element-wise adaptive optimizers, Muon uses matrix-valued updates and explicitly controls their geometry through an orthogonalization step. A central operation in Muon is the computation of the orthogonal factor $UV^\top$, where

$$W = U\Sigma V^\top$$

denotes the singular value decomposition of a matrix W . In the context of Muon, W denotes the matrix-valued update to which the orthogonalization step is applied. The factor $UV^\top$ can be interpreted as the orthogonal component of W , and it plays a key role in shaping the direction of the optimizer update [1, 2].

Computing this factor exactly by singular value decomposition is too expensive to perform repeatedly during training. Therefore, practical implementations of Muon usually approximate the orthogonalization step using Newton–Schulz iterations [1]. Newton–Schulz iteration is an iterative matrix procedure that approximates the orthogonal factor without explicitly computing a full SVD. Although this approximation is substantially cheaper than exact decomposition, it is still applied at every optimization step. For large dense matrices, repeated orthogonalization may account for a nontrivial part of the total optimizer cost. As model sizes continue to grow, this operation can become a practical bottleneck that limits the scalability of Muon [2].

1.2 Directions for improving Muon efficiency

Recent work suggests two broad directions for improving the efficiency of Muon-type optimizers. The first direction is to modify the optimization algorithm itself or the orthogonalization procedure used inside it. Scion studies training with norm-constrained linear minimization oracles and introduces normalization mechanisms that are related to the stability of orthogonalized updates [4]. Dion proposes distributed orthonormalized updates and replaces Newton–Schulz-style orthogonalization with a power-iteration-based approach that is more suitable for parallel and distributed settings [5]. Dion2 reduces the matrix size used inside Muon by applying orthogonalization to smaller randomly selected submatrices [6]. MuonBP proposes block-periodic orthogonalization, reducing the frequency or scope of expensive orthogonalization steps [7]. Other works focus more directly on improving Newton–Schulz or related matrix sign iterations, such as Gram Newton–Schulz [8] and Polar Express [9]. These methods primarily focus on changing the optimizer or reducing the cost of the matrix operation performed by the optimizer.

The second direction, which we focus on in this work, is to exploit structure in the matrix parameters themselves. Instead of applying Muon to fully dense layers, one can replace selected dense matrices with structured parameterizations that are cheaper to store, multiply, and orthogonalize. Monarch matrices are a natural candidate for this purpose: they represent a dense-like linear transformation using permutation and block-diagonal factors, and prior work has shown that such parameterizations can provide favorable efficiency–quality trade-offs in neural network training [3]. This suggests that structured layers may reduce not only the cost of the forward and backward passes, but also the cost of Muon’s Newton–Schulz orthogonalization step.

1.3 Monarch-structured layers for Muon

In this paper, we investigate whether the overhead of Muon can be reduced by replacing dense matrix layers with structured Monarch-parameterized layers [3]. Instead of assuming that an arbitrary dense matrix can be exactly decomposed into Monarch form, we restrict selected trainable matrices to a structured class of the form

$$W = PLP^\top R,$$

where P is a fixed permutation matrix, and L and R are trainable block-diagonal matrices, each consisting of two blocks. This parameterization does not represent every possible dense matrix. Rather, it replaces a dense layer with a structured surrogate that has fewer effective degrees of freedom and enables more efficient matrix operations.

This structured parameterization is particularly relevant for Muon because the expensive orthogonalization step can be applied to smaller block-diagonal factors instead of the full dense matrix. Since the blocks are independent, Newton–Schulz iterations can be performed separately for each block. This not only reduces the effective matrix size involved in orthogonalization, but also exposes additional parallelism across blocks. Therefore, Monarch-structured layers may reduce the cost of Muon both through smaller matrix operations and through more parallel execution. This idea is complementary to optimizer-level methods such as Dion, Dion2, MuonBP, Gram Newton–Schulz, and Polar Express [5–9].

1.4 Research question and contributions

The motivation for this approach is that full dense parameterization and full-matrix orthogonalization may be unnecessarily expensive for some parts of neural networks. A structured layer can preserve useful matrix interactions through permutations and block-diagonal factors while substantially reducing computational cost [3]. However, this restriction also introduces a trade-off. Since Monarch-structured layers cannot express all dense matrices, the model may lose representational flexibility, and the optimizer update may differ from that of standard Muon applied to dense layers. The main challenge is therefore to determine whether the efficiency gains outweigh the possible loss in training quality.

The central research question of this work is whether Muon can be made substantially cheaper by using Monarch-structured layers without sacrificing training performance. More specifically, we ask which structured layer designs and update strategies provide the best trade-off between computational efficiency, parallel execution, and model quality. By studying this question, we aim to clarify whether

Monarch-parameterized layers can serve as a practical mechanism for scaling Muon to larger models and more expensive training regimes.

Contributions. Our contributions can be summarized as follows:

- We propose replacing selected dense layers with Monarch-parameterized layers in order to reduce the computational cost of Muon.
- We show how the block structure of Monarch layers allows Newton–Schulz iterations to be applied to smaller independent matrices, creating opportunities for parallelization.
- We analyze the computational cost of the proposed structured Muon variants and compare them with standard Muon applied to dense layers.
- We study different strategies for updating the structured matrix factors during optimization, including block-wise updates and a random column-partitioned update inspired by Dion2 and MuonBP [6, 7].
- We identify the update strategy that provides the most favorable trade-off between computational efficiency and training performance.

Outline. The rest of the paper is organized as follows. Section 2 formalizes the problem statement and evaluation criteria. Section 3 describes the proposed structured Muon variants. Section 4 discusses related work. The following sections present the experimental setup and compare the resulting methods against Muon and AdamW baselines.

2 Problem statement

2.1 Experimental setting

We consider the problem of training a compact GPT-2 style language model for next-token prediction. The model has the following configuration:

$$n_{\text{layer}} = 4, \quad n_{\text{head}} = 4, \quad n_{\text{embd}} = 384.$$

Let

$$\mathcal{D} = \{x^{(i)}\}_{i=1}^N$$

be a dataset of token sequences, where each sequence is denoted by

$$x^{(i)} = (x_1^{(i)}, \dots, x_T^{(i)}).$$

The model f_{Θ} , parameterized by Θ , is trained to predict the next token from the previous context.

2.2 Training objective and evaluation criteria

The training objective is the standard autoregressive language modeling loss

$$\mathcal{L}(\Theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T-1} \log p_{\Theta} \left(x_{t+1}^{(i)} \mid x_{\leq t}^{(i)} \right). \quad (1)$$

We evaluate the model using validation loss and perplexity. Since perplexity is directly determined by the language modeling loss, we use

$$\text{PPL} = \exp(\mathcal{L}_{\text{val}}), \quad (2)$$

where $\mathcal{L}_{\text{val}}$ is the average validation loss.

The goal of this work is to test whether the computational cost of Muon can be reduced by replacing selected dense matrix layers in GPT-2 with Monarch-parameterized layers, while keeping the final perplexity close to the baselines. We compare the proposed structured variants with standard Muon and AdamW. The main quality requirement is that the final validation perplexity of the proposed method should not differ substantially from the perplexity obtained by the baseline optimizers. In this work, we treat a relative difference of several percent as acceptable:

$$\frac{|\text{PPL}_{\text{method}} - \text{PPL}_{\text{baseline}}|}{\text{PPL}_{\text{baseline}}} \leq \varepsilon, \quad (3)$$

where ε is chosen to be on the order of a few percent.

2.3 Standard Muon update

In standard Muon, a dense matrix parameter W is updated using a momentum matrix and an orthogonalized update [1, 2]:

$$G_t = \nabla_W \mathcal{L}(\Theta_{t-1}), \quad (4)$$

$$M_t = \beta M_{t-1} + G_t, \quad (5)$$

$$O_t = \text{NS}(M_t), \quad (6)$$

$$W_t = W_{t-1} - \alpha O_t, \quad (7)$$

where α is the learning rate, β is the momentum coefficient, and $\text{NS}(\cdot)$ denotes the Newton–Schulz approximation of the orthogonal factor. The Newton–Schulz step is applied repeatedly during training and can become expensive for large dense matrices. This motivates both optimizer-level improvements [5–9] and structure-based approaches such as the one studied in this paper.

2.4 Monarch-parameterized layers

In this work, we do not attempt to decompose an arbitrary dense matrix W into Monarch form. Instead, we replace selected dense layers with Monarch-parameterized layers [3]. For such a layer, the effective weight matrix is restricted to the structured form

$$\widetilde{W} = PLP^\top R, \quad (8)$$

where P is a fixed permutation matrix, and L and R are trainable block-diagonal matrices. This parameterization defines a restricted class of matrices and does not represent every possible dense matrix $W \in \mathbb{R}^{n \times m}$.

In our setting, each block-diagonal factor consists of two trainable blocks:

$$L = \text{diag}(L^{(1)}, L^{(2)}), \quad (9)$$

$$R = \text{diag}(R^{(1)}, R^{(2)}). \quad (10)$$

Thus, the trainable parameters of the structured layer are

$$\theta = \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\}.$$

The effective matrix $\widetilde{W}$ is determined by these four blocks, and the full set of model parameters consists of the usual GPT-2 parameters together with the structured parameters of the replaced layers.

2.5 Block-wise Muon update

Instead of applying Muon to the full effective matrix $\widetilde{W}$, we apply Muon independently to each trainable block. For any block $B \in \theta$, the update is

$$G_t^B = \nabla_B \mathcal{L}(\Theta_{t-1}), \quad (11)$$

$$M_t^B = \beta M_{t-1}^B + G_t^B, \quad (12)$$

$$O_t^B = \text{NS}(M_t^B), \quad (13)$$

$$B_t = B_{t-1} - \alpha O_t^B. \quad (14)$$

After updating all four blocks, the effective layer matrix is reconstructed as

$$\widetilde{W}_t = PL_t P^\top R_t. \quad (15)$$

2.6 Desired trade-off

The central problem studied in this work is whether this structured replacement can reduce the cost of Muon’s Newton–Schulz orthogonalization while preserving the training quality of the original model. The expected benefit is that Newton–Schulz iterations are applied to smaller block matrices instead of a full dense matrix, and these block-wise computations can be performed independently. The main risk is that Monarch-parameterized layers are less expressive than dense layers, which may increase validation loss and perplexity.

Therefore, the desired solution is a Monarch-based Muon variant that achieves a favorable trade-off between computational efficiency and language modeling quality. In particular, we seek a strategy whose final validation loss and perplexity remain within a few percent of the corresponding Muon or AdamW baselines, while reducing the cost of the orthogonalization step through smaller and potentially parallel block-wise Newton–Schulz updates.

3 Theory

3.1 Monarch-parameterized model

We consider a selected dense linear layer with weight matrix W . Instead of training W as a fully dense matrix, we replace it with a Monarch-parameterized matrix [3]

$$\widetilde{W} = PLP^\top R, \quad (16)$$

where P is a fixed permutation matrix, and L and R are trainable block-diagonal matrices. In our setting, each of these matrices consists of two trainable blocks:

$$L = \text{diag}(L^{(1)}, L^{(2)}), \quad (17)$$

$$R = \text{diag}(R^{(1)}, R^{(2)}). \quad (18)$$

Thus, the trainable parameters of one Monarch layer are

$$\theta = \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\}.$$

This parameterization restricts the class of possible weight matrices. In other words, $\widetilde{W}$ is not an arbitrary dense matrix, and not every dense matrix W can be represented in this form. However, the structure preserves interactions between different coordinate groups through the permutation matrix P , while the block-diagonal factors make matrix operations cheaper and more suitable for parallel computation [3].

The main idea of our method is to apply Muon not to the full effective matrix $\widetilde{W}$, but to the smaller trainable blocks $L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}$. This changes the orthogonalization problem from one large Newton–Schulz computation to several smaller independent computations.

3.2 Newton–Schulz orthogonalization

Muon uses an approximate orthogonalized update. Given a matrix A , we define the Newton–Schulz approximation as follows [1, 2]. First, the matrix is normalized:

$$X_0 = \frac{A}{\|A\|_F}. \quad (19)$$

Then, for $k = 0, \dots, K - 1$, we apply

$$S_k = X_k X_k^\top, \quad (20)$$

$$X_{k+1} = aX_k - bS_k X_k + cS_k^2 X_k, \quad (21)$$

where in our experiments we use

$$K = 5, \quad (a, b, c) = (3.4445, 4.7750, 2.0315).$$

The final approximation is

$$\text{NS}(A) = X_K. \quad (22)$$

In standard Muon, this procedure is applied to a full momentum matrix. In the proposed method, it is applied separately to each Monarch block. Alternative improvements of the same general orthogonalization component have been studied in Gram Newton–Schulz and Polar Express [8, 9].

3.3 Block-wise Muon update

For each trainable block

$$B \in \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\},$$

we maintain a separate momentum matrix M_t^B . At training step t , the gradient with respect to block B is

$$G_t^B = \nabla_B \mathcal{L}(\Theta_{t-1}), \quad (23)$$

where Θ denotes all trainable model parameters. The block-wise Muon update is then given by

$$M_t^B = \beta M_{t-1}^B + G_t^B, \quad (24)$$

$$O_t^B = \text{NS}(M_t^B), \quad (25)$$

$$B_t = B_{t-1} - \alpha O_t^B, \quad (26)$$

where α is the learning rate and β is the momentum coefficient.

After all four blocks are updated, the effective layer matrix is reconstructed as

$$\widetilde{W}_t = PL_t P^\top R_t. \quad (27)$$

Thus, the optimizer never needs to apply Newton–Schulz iterations to the full dense effective matrix $\widetilde{W}_t$. Instead, it orthogonalizes four smaller matrices independently. This structure-based reduction is complementary to methods that shrink or periodically orthogonalize matrices inside the optimizer itself [6, 7].

3.4 Algorithm

The full training procedure for a Monarch-parameterized layer with block-wise Muon updates is shown in Algorithm 1.

Algorithm 1 Block-wise Muon for Monarch-parameterized layers

Require: Dataset $\mathcal{D}$, model f_Θ , learning rate α , momentum coefficient β , Newton–Schulz iterations K

Require: Monarch blocks $L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}$ constructed by the Monarch layer

```

1:  $\mathcal{B} \leftarrow \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\}$ 
2: Initialize  $M_0^B \leftarrow 0$  for all  $B \in \mathcal{B}$ 
3: for training step  $t = 1, 2, \dots$  do
4:   Sample a mini-batch from  $\mathcal{D}$ 
5:   Construct  $\widetilde{W}_{t-1} = PL_{t-1}P^\top R_{t-1}$ 
6:   Compute  $\mathcal{L}(\Theta_{t-1})$  and gradients by backpropagation
7:   for each block  $B \in \mathcal{B}$  do
8:      $G_t^B \leftarrow \nabla_B \mathcal{L}(\Theta_{t-1})$ 
9:      $M_t^B \leftarrow \beta M_{t-1}^B + G_t^B$ 
10:     $B_t \leftarrow B_{t-1} - \alpha \text{NS}_K(M_t^B)$ 
11:   end for
12:   Update non-Monarch parameters according to the chosen experimental protocol
13: end for
```

3.5 Dion2-inspired random column-partitioned update

In addition to the purely block-wise Muon update, we consider a periodic random column-partitioned strategy inspired by Dion2 and MuonBP [6, 7]. Dion2 motivates reducing the cost of Muon by applying orthogonalization to smaller randomly selected parts of a matrix, while MuonBP motivates periodic variants of orthogonalization. In our setting, we adapt these ideas to Monarch-parameterized layers. The goal is to introduce partial coupling between the two blocks of each Monarch factor while still avoiding Newton–Schulz orthogonalization of the full effective matrix.

For each Monarch factor, we concatenate its two trainable blocks along the column dimension according to the Monarch layer layout. For the L -factor, this gives

$$C_L = \text{ColConcat}\left(L^{(1)}, L^{(2)}\right),$$

and for the corresponding momentum matrices,

$$M_L = \text{ColConcat}(M^{L^{(1)}}, M^{L^{(2)}}).$$

At every periodic step, we randomly sample half of the columns of C_L . Let this random set of column indices be denoted by $\mathcal{I}_L$, and let $\bar{\mathcal{I}}_L$ be its complement. We then apply Newton–Schulz orthogonalization separately to the selected and unselected column groups:

$$O_L^{\mathcal{I}} = \text{NS}(M_L[:, \mathcal{I}_L]), \quad O_L^{\bar{\mathcal{I}}} = \text{NS}(M_L[:, \bar{\mathcal{I}}_L]).$$

The two column groups are updated independently, scattered back into the concatenated matrix C_L , and then split back into $L^{(1)}$ and $L^{(2)}$ using the original Monarch block boundaries. The same random column-partitioned procedure is applied to the R -factor.

Here, $\text{ColSplit}_{\text{orig}}$ denotes splitting a concatenated matrix back according to the original Monarch block boundaries.

This strategy differs from the purely block-wise update because the random column split changes over training. As a result, different columns from the original Monarch blocks are periodically grouped together, allowing the optimizer to mix information across blocks over time. At the same time, each Newton–Schulz computation remains smaller than the full dense update. Thus, the method combines the structure of Monarch matrices [3] with ideas from recent Muon variants that shrink or periodically modify the orthogonalization step [6, 7].

Algorithm 2 Random column-partitioned Muon update for Monarch layers

Require: Learning rate α , momentum coefficient β , period q

Require: Monarch blocks $L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}$ constructed by the Monarch layer

```

1: Initialize  $M_0^B \leftarrow 0$  for all  $B \in \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\}$ 
2: for training step  $t = 1, 2, \dots$  do
3:   Compute loss  $\mathcal{L}(\Theta_{t-1})$  and gradients by backpropagation
4:   for each block  $B \in \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\}$  do
5:      $G_t^B \leftarrow \nabla_B \mathcal{L}(\Theta_{t-1})$ 
6:      $M_t^B \leftarrow \beta M_{t-1}^B + G_t^B$ 
7:   end for
8:   if  $t \bmod q = 0$  then
9:     for each factor  $F \in \{L, R\}$  do
10:       $C_F \leftarrow \text{ColConcat}(F_{t-1}^{(1)}, F_{t-1}^{(2)})$ 
11:       $M_F \leftarrow \text{ColConcat}(M_t^{F^{(1)}}, M_t^{F^{(2)}})$ 
12:      Sample random  $\mathcal{I}_F$  such that

```

$$|\mathcal{I}_F| = \frac{1}{2} \text{cols}(C_F)$$

```

13:      Let  $\bar{\mathcal{I}}_F$  be the complement of  $\mathcal{I}_F$ 
14:       $C_F[:, \mathcal{I}_F] \leftarrow C_F[:, \mathcal{I}_F] - \alpha \text{NS}_K(M_F[:, \mathcal{I}_F])$ 
15:       $C_F[:, \bar{\mathcal{I}}_F] \leftarrow C_F[:, \bar{\mathcal{I}}_F] - \alpha \text{NS}_K(M_F[:, \bar{\mathcal{I}}_F])$ 
16:       $(F_t^{(1)}, F_t^{(2)}) \leftarrow \text{ColSplit}_{\text{orig}}(C_F)$ 
17:    end for
18:  else
19:    for each block  $B \in \{L^{(1)}, L^{(2)}, R^{(1)}, R^{(2)}\}$  do
20:       $B_t \leftarrow B_{t-1} - \alpha \text{NS}_K(M_t^B)$ 
21:    end for
22:  end if
23:  Reconstruct  $\widetilde{W}_t = PL_t P^\top R_t$ 
24: end for

```

3.6 Properties of the proposed algorithms

The proposed algorithms have several useful properties. First, they reduce the size of the matrices used in Newton–Schulz orthogonalization. Standard Muon applies Newton–Schulz iterations to full dense matrix updates [1, 2]. In contrast, the proposed block-wise variant applies the same procedure

to smaller Monarch factors. Since Newton–Schulz iterations rely on matrix multiplications, reducing matrix size directly reduces the computational cost of the orthogonalization step.

Second, the block-wise Newton–Schulz computations are independent after gradients have been computed. Therefore, the updates of $L^{(1)}$, $L^{(2)}$, $R^{(1)}$, and $R^{(2)}$ can be performed in parallel. This parallels the motivation behind distributed or parallel orthogonalized-update methods such as Dion [5], but the source of parallelism in our method comes from the structured parameterization rather than from a distributed reformulation of the optimizer.

Third, the random column-partitioned update introduces additional coupling between the two blocks of each Monarch factor. This is inspired by the idea of shrinking the matrix inside Muon as in Dion2 [6] and by periodic orthogonalization schemes such as MuonBP [7]. The random partition changes over time, so different columns are grouped together across training steps.

Finally, the proposed method changes the optimization problem. The model no longer optimizes over all dense matrices W , but over structured parameters $L^{(1)}$, $L^{(2)}$, $R^{(1)}$, and $R^{(2)}$. This can reduce computational cost, but it may also reduce expressivity. Therefore, the method introduces a trade-off between efficiency and model quality, similar to the general trade-offs studied in structured matrix parameterizations [3].

4 Related Work

4.1 Muon and orthogonalized optimizers

Muon was introduced as an optimizer that applies orthogonalized updates to matrix-valued parameters, with practical implementations relying on Newton–Schulz iterations to approximate the polar factor [1]. Recent work demonstrated that Muon can be scaled to large language model training when combined with suitable update scaling and weight decay choices [2]. This result motivates the study of Muon as a competitive alternative to standard adaptive optimizers such as AdamW, while also highlighting the importance of reducing the additional cost introduced by orthogonalization.

Several works improve or modify Muon-type updates. Scion studies training with norm-constrained linear minimization oracles and introduces normalization mechanisms that are related to the stability of orthogonalized updates [4]. Dion proposes distributed orthonormalized updates and replaces Newton–Schulz-style orthogonalization with a power-iteration-based approach that is better suited for parallel execution [5]. Dion2 reduces the size of the matrix used in Muon by applying orthogonalization to smaller randomly selected submatrices [6]. MuonBP proposes block-periodic orthogonalization, reducing the frequency or scope of expensive orthogonalization steps [7]. In contrast to these works, our method does not primarily modify Muon itself, but changes the structure of the trainable matrices to make the orthogonalization step cheaper.

4.2 Improving Newton–Schulz orthogonalization

Another line of work focuses directly on improving the Newton–Schulz iteration or related matrix sign methods. Gram Newton–Schulz proposes a modified formulation of the Newton–Schulz procedure for Muon-style optimization [8]. Polar Express studies optimal matrix sign methods and their application to the Muon algorithm [9]. These methods are complementary to our approach: they aim to make the orthogonalization procedure itself more efficient or more accurate, whereas we reduce the size of the matrices to which orthogonalization is applied.

4.3 Structured matrix parameterizations

Structured matrices provide a different way to reduce computational cost by restricting the class of trainable weight matrices. Monarch matrices represent a dense-like linear transformation as a product of permutation and block-diagonal matrices and were shown to provide efficient and accurate training in neural networks [3]. Our work builds on this representation by using Monarch-parameterized layers as a mechanism for reducing the cost of Muon. Instead of applying Newton–Schulz iterations to a full dense matrix update, we apply them to smaller block-structured factors, which can reduce computational cost and expose additional parallelism.

5 Preliminaries

5.1 General notation

In this section, we introduce the general notation used in the rest of the paper. We denote matrices by uppercase letters, vectors by lowercase letters, and trainable model parameters by Θ . For a matrix A , $\|A\|_F$ denotes its Frobenius norm. The notation $\text{NS}_K(A)$ denotes the result of applying K Newton–Schulz iterations to the matrix A . For a matrix C , the notation $C[:, \mathcal{I}]$ denotes the submatrix formed by selecting the columns indexed by $\mathcal{I}$.

5.2 Monarch layer structure

For each selected dense layer, we replace the original weight matrix with a Monarch-parameterized matrix

$$\widetilde{W} = PLP^\top R,$$

where P is a fixed permutation matrix, and L and R are trainable block-diagonal factors. In our implementation, each factor consists of two trainable blocks:

$$L = \text{diag}(L^{(1)}, L^{(2)}), \quad R = \text{diag}(R^{(1)}, R^{(2)}).$$

The dimensions of these blocks are determined by the Monarch layer construction and by the input and output dimensions of the replaced dense layer. They are not additional assumptions about the data, the model distribution, or the optimal dense weight matrix. Rather, they are part of the structured architecture used in place of the dense layer.

For the random column-partitioned update, we use the block layout produced by the Monarch parameterization. The two blocks of each factor are concatenated along the column dimension according to this layout, updated, and then split back using the original block boundaries. Thus, the splitting operation is determined by the structured layer construction, not by an additional equal-size-block assumption.

6 Method

The proposed method replaces selected dense layers of a compact GPT-2 style model with Monarch-parameterized layers [3]. The resulting trainable parameters are the block factors $L^{(1)}$, $L^{(2)}$, $R^{(1)}$, and $R^{(2)}$. We then apply Muon-style orthogonalized updates to these factors rather than to the full effective matrix $\widetilde{W}$.

We study two update strategies. The first is the purely block-wise strategy, where each block is updated independently using Newton–Schulz orthogonalization. The second is the random column-partitioned strategy, inspired by Dion2 and MuonBP [6, 7], where the blocks of each Monarch factor are periodically concatenated according to the Monarch layout, randomly partitioned by columns, orthogonalized in two groups, and then split back using the original block boundaries.

7 Experiments

To verify the theoretical estimates obtained, we conducted a detailed empirical study. We train a compact GPT-2 style language model with

$$n_{\text{layer}} = 4, \quad n_{\text{head}} = 4, \quad n_{\text{embd}} = 384.$$

We compare AdamW, standard Muon, block-wise Monarch Muon, and the random column-partitioned Monarch Muon variant. Following the motivation of Muon as a scalable optimizer for language model training [2], we use validation loss and perplexity as the main quality metrics. We also track computational characteristics related to the Newton–Schulz step, since this is the main source of additional cost in Muon-style methods [1, 8, 9].

The proposed method is considered successful if its final perplexity differs from the Muon or AdamW baseline by only a few percent while reducing the cost of orthogonalization through smaller and potentially parallel block-wise Newton–Schulz updates.

8 Discussion

The proposed approach is complementary to recent optimizer-level improvements of Muon. Methods such as Dion, Dion2, MuonBP, Gram Newton–Schulz, and Polar Express modify the orthogonalization procedure, reduce the matrix size inside the optimizer, or improve the underlying matrix iteration [5–9]. In contrast, our method changes the parameterization of the model itself by introducing Monarch-structured layers [3]. This means that the computational gain comes from applying the same Muon principle to smaller structured factors.

At the same time, the method introduces a restriction on the hypothesis class of the model. A Monarch-parameterized layer cannot represent all dense matrices, so the main empirical question is whether the reduction in expressivity leads to a noticeable degradation in validation loss or perplexity. If the degradation remains small, the method can provide a practical way to reduce Muon’s cost in settings where full dense orthogonalization is too expensive.

9 Conclusion

We studied a structure-based approach to reducing the computational cost of Muon. Instead of applying Muon to full dense layers, we replace selected layers with Monarch-parameterized matrices and apply Newton–Schulz orthogonalization to smaller trainable blocks. We also considered a random column-partitioned update inspired by Dion2 and MuonBP, which periodically couples columns across the two blocks of each Monarch factor. The resulting methods aim to preserve the optimization benefits of Muon while reducing the cost of its most expensive operation.

References

- [1] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- [2] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is scalable for llm training, 2025. URL <https://arxiv.org/abs/2502.16982>.
- [3] Tri Dao, Beidi Chen, Nimit Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. Monarch: Expressive structured matrices for efficient and accurate training, 2022. URL <https://arxiv.org/abs/2204.00595>.
- [4] Thomas Pethick, Wanyun Xie, Kimon Antonakopoulos, Zhenyu Zhu, Antonio Silveti-Falls, and Volkan Cevher. Training deep learning models with norm-constrained lmos, 2025. URL <https://arxiv.org/abs/2502.07529>.
- [5] Kwangjun Ahn, Byron Xu, Natalie Abreu, Ying Fan, Gagik Magakyan, Pratyusha Sharma, Zheng Zhan, and John Langford. Dion: Distributed orthonormalized updates, 2025. URL <https://arxiv.org/abs/2504.05295>.
- [6] Kwangjun Ahn, Noah Amsel, and John Langford. Dion2: A simple method to shrink matrix in muon, 2025. URL <https://arxiv.org/abs/2512.16928>.
- [7] Ahmed Khaled, Kaan Ozkara, Tao Yu, Mingyi Hong, and Youngsuk Park. Muonbp: Faster muon via block-periodic orthogonalization, 2025. URL <https://arxiv.org/abs/2510.16981>.
- [8] Jack Zhang, Noah Amsel, Berlin Chen, and Tri Dao. Gram newton-schulz, 2026. URL <https://dao-ailab.github.io/blog/2026/gram-newton-schulz/>.
- [9] Noah Amsel, David Persson, Christopher Musco, and Robert M. Gower. The polar express: Optimal matrix sign methods and their application to the muon algorithm, 2026. URL <https://arxiv.org/abs/2505.16932>.

- [10] Tom M Mitchell. The need for biases in learning generalizations. 1980.
- [11] Jeremy Bernstein and Laker Newhouse. Old optimizer, new norm: An anthology, 2024. URL <https://arxiv.org/abs/2409.20325>.
- [12] Jiaxiang Li and Mingyi Hong. A note on the convergence of muon, 2025. URL <https://arxiv.org/abs/2502.02900>.
- [13] Alexey Kravatskiy, Ivan Kozyrev, Nikolai Kozlov, Alexander Vinogradov, Daniil Merkulov, and Ivan Oseledets. The ky fan norms and beyond: Dual norms and combinations for matrix optimization, 2025. URL <https://arxiv.org/abs/2512.09678>.
- [14] Michael Crawshaw, Chirag Modi, Mingrui Liu, and Robert M. Gower. An exploration of non-euclidean gradient descent: Muon and its many variants, 2025. URL <https://arxiv.org/abs/2510.09827>.
- [15] Kaja Grunkowska, Alexander Gaponov, Zhirayr Tovmasyan, and Peter Richtárik. Error feedback for muon and friends, 2025. URL <https://arxiv.org/abs/2510.00643>.
- [16] Aman Gupta, Rafael Celente, Abhishek Shivanna, D. T. Braithwaite, Gregory Dexter, Shao Tang, Hiroto Udagawa, Daniel Silva, Rohan Ramanath, and S. Sathiya Keerthi. Effective quantization of muon optimizer states, 2026. URL <https://arxiv.org/abs/2509.23106>.
- [17] Dmitry Kovalev. Understanding gradient orthogonalization for deep learning via non-euclidean trust-region optimization, 2025. URL <https://arxiv.org/abs/2503.12645>.

A Appendix / supplemental material

A.1 Additional experiments / Proofs of Theorems

TODO